It's rapidly gaining traction because it simplifies the process of extending applications with external functionality and enables interoperability across ecosystems.

Any client that implements this protocol, whether a server application, browser-based web app, or CLI tool, can interact consistently with MCP servers. This means the AI agent, wherever it runs, always knows how to call a tool and handle its outputs, whether that tool accesses an API, a database, or a filesystem. In short, adding MCP capabilities to your applications allows them to tap into the many tools and databases that others create, provided those tools are also MCP compatible.

The MCP also supports scalable architectures, allowing multiple external services to be connected via MCP servers. For instance, our React/Next.js web app can communicate with one or more MCP servers, each providing different tools, without rewriting the assistant's core logic. The MCP can be applied to any client or application/server-side apps, agentic systems, or CLI tools.

In practice, the MCP lets developers define tools once and reuse them across AI systems. This separation of tools from assistants simplifies integration, enforces consistent security and resource management, and makes it easier to scale AI applications

In this chapter, we build and integrate a complete MCP server into a full-stack AI application using the Vercel AI SDK, then see how this pattern can scale to multiple servers working together. The goal is to give you a clear starting point for adopting the MCP in your own projects.

12.1 Why the MCP matters for AI integration

Before we get to the practical examples, let's revisit some of the core concepts we've explored in this book: prompts, tools, and generated resources. These form the foundation of modern AI applications and help us understand why a standard such as the MCP is needed.

Prompts, as you may recall, are the structured inputs we provide to large language models (LLMs). Prompts shape the model's behavior, control its reasoning, and define the context in which the AI operates.

Tools are the external capabilities that extend what an AI model can do beyond generating text—such as file access, database queries, or API calls. Tools can run on the client side (invoked by an AI agent in the application) or the server side (invoked by an LLM or MCP server) or be shared across systems. In more complex setups, multiple agents or models may each access different tools, combining their results into a coordinated workflow.

Generated content is the output created by the model or produced from tool usage. These outputs can be natural language responses, structured data, summaries, or even programmatic results like SQL query outputs.

As we've seen, prompts control AI behavior, tools extend its capabilities, and resources are generated to create value. Each integration requires custom wiring: developers define how tools are exposed, how the AI calls them, and how results flow back. Different frameworks approach this differently: Astra, our browser-based chat app built with React and Next.js, calls tools either from the web server or via LLMs like Gemini